МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”

Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по рубежному контролю №2

Выполнил:
Студент группы ИУ5-34Б
Яхутль М. А.
Преподаватель:
Гапанюк Ю. Е.

Москва 2025

Код rk2.py

```
class Klass:
    def __init__(self, klass_id: int, name: str):
        self.klass_id = klass_id
        self.name = name

    def __repr__(self):
        return f"Klass({self.klass_id}, '{self.name}')"


class Schoolchild:
    def __init__(self, child_id: int, name: str, grade: float):
        self.child_id = child_id
        self.name = name
        self.grade = grade

    def __repr__(self):
        return f"Schoolchild({self.child_id}, '{self.name}', {self.grade})"


class SchoolchildKlass:
    def __init__(self, child_id: int, klass_id: int):
        self.child_id = child_id
        self.klass_id = klass_id

    def __repr__(self):
        return f"SchoolchildKlass(child_id={self.child_id}, klass_id={self.klass_id})"


def query_1(klasses_list, schoolchildren_list, schoolchild_klasses_list):
    classes_starting_with_A = [k for k in klasses_list if k.name.startswith('A')]

    children_in_A_classes = []
    for j in schoolchild_klasses_list:
        if any(k.klass_id == j.klass_id for k in classes_starting_with_A):
            child = next(c for c in schoolchildren_list if c.child_id == j.child_id)
            children_in_A_classes.append((child, j.klass_id))

    return children_in_A_classes, classes_starting_with_A


def query_2(schoolchildren_list, schoolchild_klasses_list):
    klass_grades = {}
    for sk in schoolchild_klasses_list:
        klass_id = sk.klass_id
        child = next(c for c in schoolchildren_list if c.child_id == sk.child_id)
        if klass_id not in klass_grades:
            klass_grades[klass_id] = []
        klass_grades[klass_id].append(child.grade)

    klass_avg_grades = {
        klass_id: sum(grades) / len(grades)
        for klass_id, grades in klass_grades.items()
    }

    sorted_klasses = sorted(
        klass_avg_grades.items(),
        key=lambda x: x[1],
```

```
        reverse=True
    )

    return sorted_klasses


def query_3(klasses_list, schoolchildren_list, schoolchild_klasses_list):
    associations = []
    for sk in schoolchild_klasses_list:
        klass = next(k for k in klasses_list if k.klass_id == sk.klass_id)
        child = next(c for c in schoolchildren_list if c.child_id == sk.child_id)
        associations.append((klass, child))

    associations_sorted = sorted(associations, key=lambda x: x[0].name)

    return associations_sorted


def print_query_1_result(children_in_A_classes, classes_starting_with_A, klasses_list):
    print("=== Запрос 1 ===")
    print("Классы, начинающиеся с 'A':")
    for klass in classes_starting_with_A:
        print(f"  - {klass.name}")

    print("\nШкольники в этих классах:")
    for child, klass_id in children_in_A_classes:
        klass_name = next(k.name for k in klasses_list if k.klass_id == klass_id)
        print(f"  - {child.name} (класс: {klass_name})")


def print_query_2_result(sorted_klasses, klasses_list):
    print("\n=== Запрос 2 ===")
    print("Классы, отсортированные по среднему баллу (убывание):")
    for klass_id, avg_grade in sorted_klasses:
        klass_name = next(k.name for k in klasses_list if k.klass_id == klass_id)
        print(f"  - {klass_name}: средний балл = {avg_grade:.2f}")


def print_query_3_result(associations_sorted):
    print("\n=== Запрос 3 ===")
    print("Связи школьников и классов (отсортировано по классам):")
    for klass, child in associations_sorted:
        print(f"  - Класс: {klass.name}, Школьник: {child.name}")


def main():
    klasses = [
        Klass(1, "A11"),
        Klass(2, "Б10"),
        Klass(3, "A9"),
        Klass(4, "Б8"),
        Klass(5, "B7")
    ]

    schoolchildren = [
        Schoolchild(1, "Иванова Мария", 4.5),
        Schoolchild(2, "Петров Алексей", 4.8),
        Schoolchild(3, "Сидорова Анна", 4.2),
        Schoolchild(4, "Козлов Михаил", 4.9),
```

```
        Schoolchild(5, "Морозова Елена", 4.6),
        Schoolchild(6, "Лебедев Роман", 4.3),
        Schoolchild(7, "Федорова Ольга", 4.7),
        Schoolchild(8, "Григорьев Денис", 4.1),
    ]

    schoolchild_klasses = [
        SchoolchildKlass(1, 1),
        SchoolchildKlass(2, 1),
        SchoolchildKlass(3, 2),
        SchoolchildKlass(4, 2),
        SchoolchildKlass(5, 3),
        SchoolchildKlass(6, 3),
        SchoolchildKlass(7, 4),
        SchoolchildKlass(8, 5),
    ]

    print("=== Программа: Школьник — Класс ===\n")

    result1, classes_A = query_1(klasses, schoolchildren, schoolchild_klasses)
    print_query_1_result(result1, classes_A, klasses)

    result2 = query_2(schoolchildren, schoolchild_klasses)
    print_query_2_result(result2, klasses)

    result3 = query_3(klasses, schoolchildren, schoolchild_klasses)
    print_query_3_result(result3)


if __name__ == "__main__":
    main()
```

Код rk2_tests.py

```
import unittest
from rk2 import Klass, Schoolchild, SchoolchildKlass, query_1, query_2, query_3

class TestSchoolSystem(unittest.TestCase):
    def setUp(self):
        self.klasses = [
            Klass(1, "А11"),
            Klass(2, "Б10"),
            Klass(3, "А9"),
            Klass(4, "Б8"),
            Klass(5, "В7")
        ]

        self.schoolchildren = [
            Schoolchild(1, "Иванова Мария", 4.5),
            Schoolchild(2, "Петров Алексей", 4.8),
            Schoolchild(3, "Сидорова Анна", 4.2),
            Schoolchild(4, "Козлов Михаил", 4.9),
```

```
            Schoolchild(5, "Морозова Елена", 4.6),
            Schoolchild(6, "Лебедев Роман", 4.3),
            Schoolchild(7, "Федорова Ольга", 4.7),
            Schoolchild(8, "Григорьев Денис", 4.1),
        ]

        self.schoolchild_klasses = [
            SchoolchildKlass(1, 1),
            SchoolchildKlass(2, 1),
            SchoolchildKlass(3, 2),
            SchoolchildKlass(4, 2),
            SchoolchildKlass(5, 3),
            SchoolchildKlass(6, 3),
            SchoolchildKlass(7, 4),
            SchoolchildKlass(8, 5),
        ]

    def test_query_1(self):
        children_in_A, classes_A = query_1(self.klasses, self.schoolchildren,
self.schoolchild_klasses)

        # Проверяем, что найдены правильные классы на 'A'
        self.assertEqual(len(classes_A), 2)  # A11 и A9
        self.assertEqual(classes_A[0].name, "A11")
        self.assertEqual(classes_A[1].name, "A9")

        # Проверяем, что найдены правильные школьники
        self.assertEqual(len(children_in_A), 4)  # 4 школьника в классах A11 и A9

        # Проверяем конкретных школьников
        student_names = [child.name for child, _ in children_in_A]
        self.assertIn("Иванова Мария", student_names)
        self.assertIn("Петров Алексей", student_names)
        self.assertIn("Морозова Елена", student_names)
        self.assertIn("Лебедев Роман", student_names)

    def test_query_2(self):
        sorted_grades = query_2(self.schoolchildren, self.schoolchild_klasses)

        self.assertEqual(len(sorted_grades), 5)  # 5 классов

        grades = [avg for _, avg in sorted_grades]
        self.assertTrue(all(grades[i] >= grades[i + 1] for i in range(len(grades) - 1)))

        #A11: Иванова (4.5) и Петров (4.8) => среднее = (4.5 + 4.8) / 2 = 4.65
        klass_id_A11 = 1
        for klass_id, avg_grade in sorted_grades:
```

```
            if klass_id == klass_id_A11:
                self.assertAlmostEqual(avg_grade, 4.65, places=2)
                break

    def test_query_3(self):
        associations = query_3(self.klasses, self.schoolchildren,
self.schoolchild_klasses)

        self.assertEqual(len(associations), 8)  # 8 школьников

        class_names = [klass.name for klass, _ in associations]
        self.assertEqual(class_names, ["А11", "А11", "А9", "А9", "Б10", "Б10",
"Б8", "В7"])

        first_association = associations[0]
        self.assertEqual(first_association[0].name, "А11")  # Класс
        self.assertEqual(first_association[1].name, "Иванова Мария")  # Школьник

    def test_empty_data(self):
        empty_result1, empty_classes_A = query_1([], [], [])
        self.assertEqual(len(empty_result1), 0)
        self.assertEqual(len(empty_classes_A), 0)

        empty_result2 = query_2([], [])
        self.assertEqual(len(empty_result2), 0)

        empty_result3 = query_3([], [], [])
        self.assertEqual(len(empty_result3), 0)


if __name__ == "__main__":
    unittest.main()
```

Результат выполнения:

```
Ran 4 tests in 0.002s

OK

Process finished with exit code 0
```